



US 20250285380A1

(19) **United States**

(12) **Patent Application Publication**
Kulon et al.

(10) **Pub. No.: US 2025/0285380 A1**

(43) **Pub. Date: Sep. 11, 2025**

(54) **GENERATING THREE-DIMENSIONAL
OBJECT MODELS FROM
TWO-DIMENSIONAL IMAGES**

(71) Applicant: **Snap Inc.**, Santa Monica, CA (US)

(72) Inventors: **Dominik Kulon**, London (GB); **Riza
Alp Guler**, London (GB); **Iason
Kokkinos**, London (GB); **Stefanos
Zafeiriou**, London (GB)

(21) Appl. No.: **19/214,779**

(22) Filed: **May 21, 2025**

Related U.S. Application Data

(63) Continuation of application No. 17/760,424, filed on
Aug. 9, 2022, filed as application No. PCT/GB2020/
050371 on Feb. 17, 2020, now Pat. No. 12,340,467.

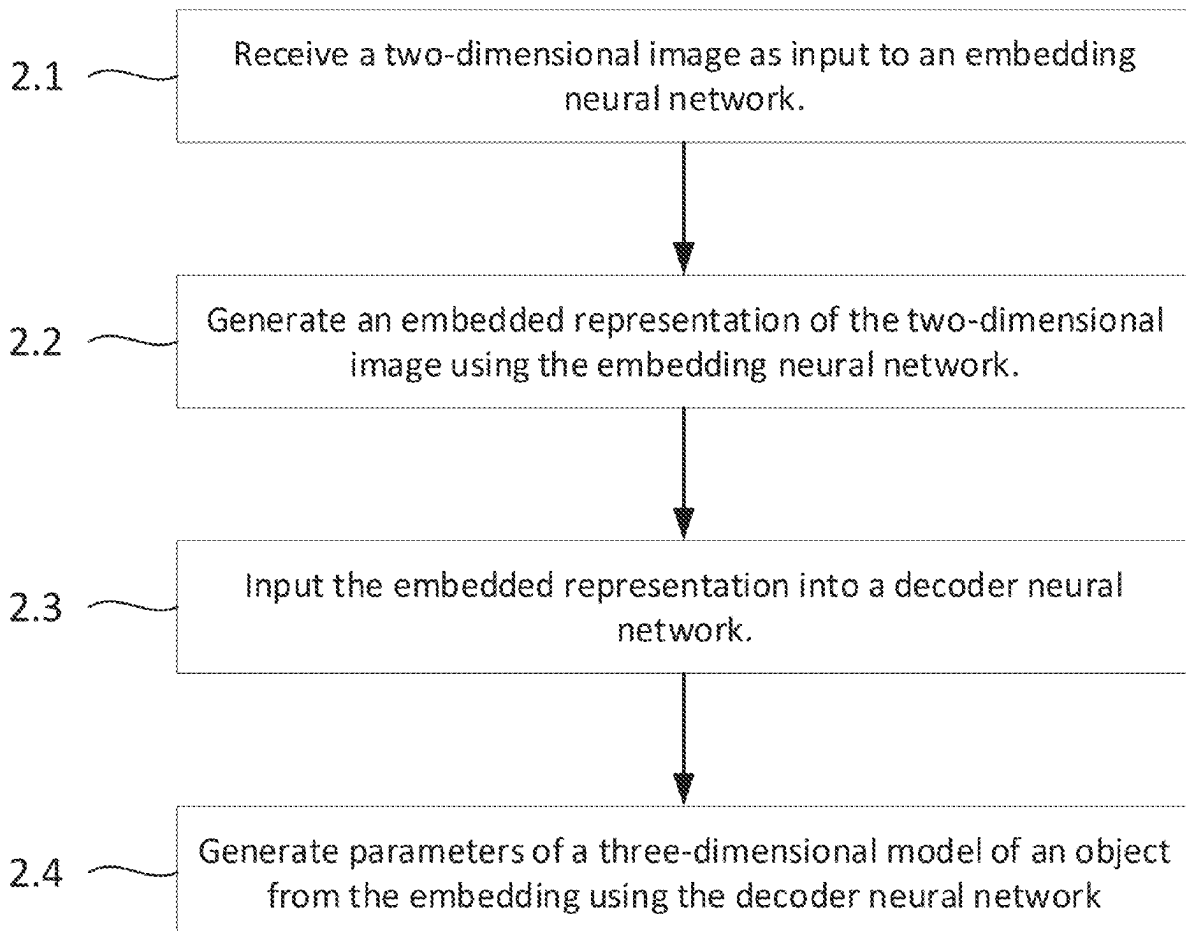
Publication Classification

(51) **Int. Cl.**
G06T 17/20 (2006.01)
G06T 3/40 (2024.01)
G06T 7/00 (2017.01)

(52) **U.S. Cl.**
CPC **G06T 17/20** (2013.01); **G06T 3/40**
(2013.01); **G06T 7/97** (2017.01); **G06T**
2207/20081 (2013.01); **G06T 2207/20084**
(2013.01)

(57) **ABSTRACT**

This specification discloses methods and systems for generating three-dimensional models of deformable objects from two-dimensional images. According to one aspect of this disclosure, there is described a computer implemented method for generating a three dimensional model of deformable object from a two-dimensional image. The method comprises: receiving, as input to an embedding neural network, the two-dimensional image, wherein the two dimensional image comprises an image of an object; generating, using the embedding neural network, an embedded representation of a two-dimensional image; inputting the embedded representation into a learned decoder model; and generating, using the learned decoder model, parameters of the three dimensional model of the object from the embedded representation.



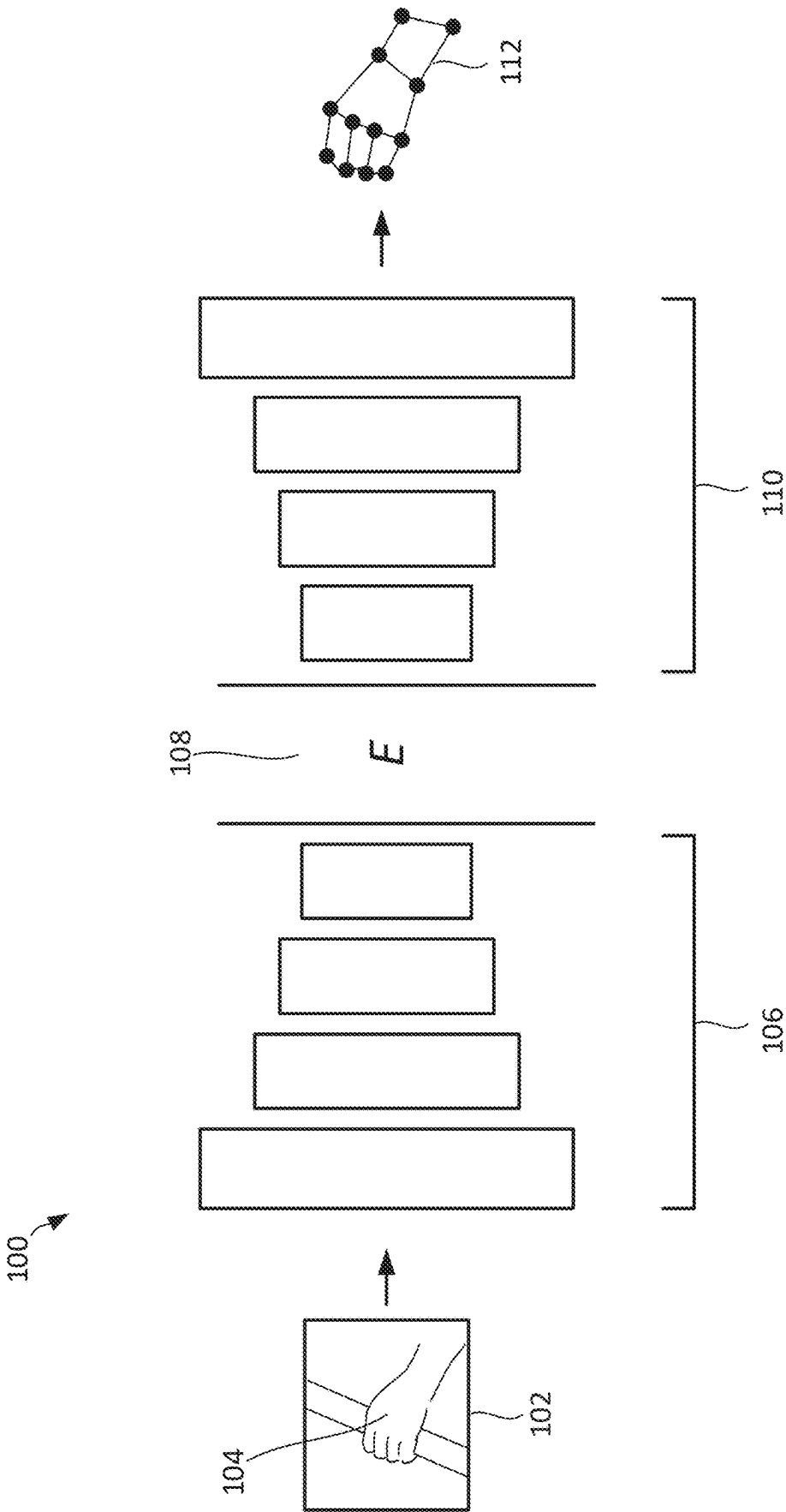


FIG. 1

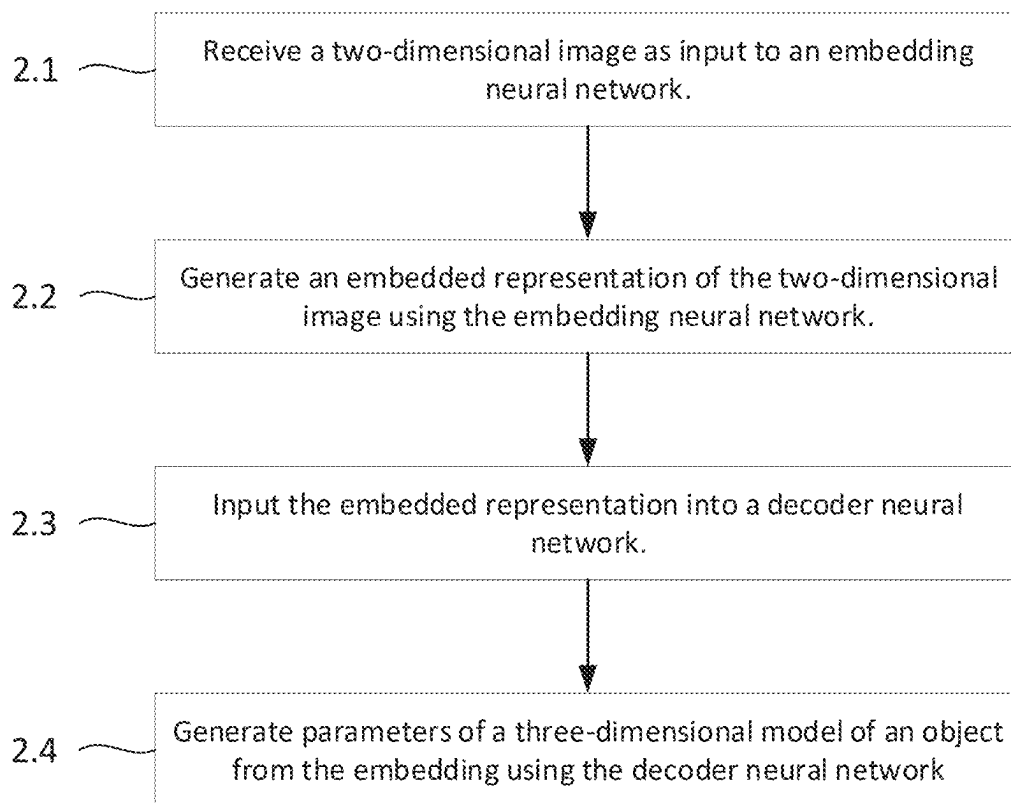


FIG. 2

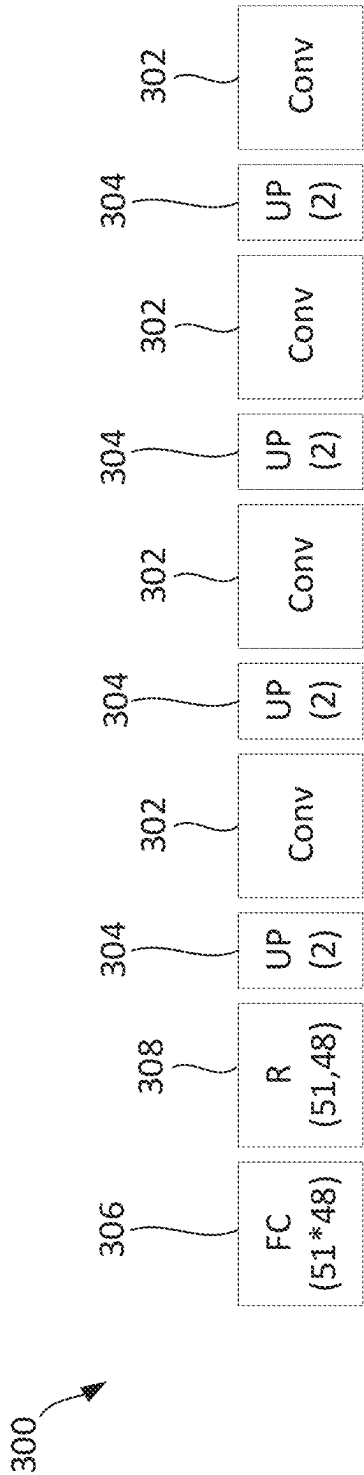


FIG. 3

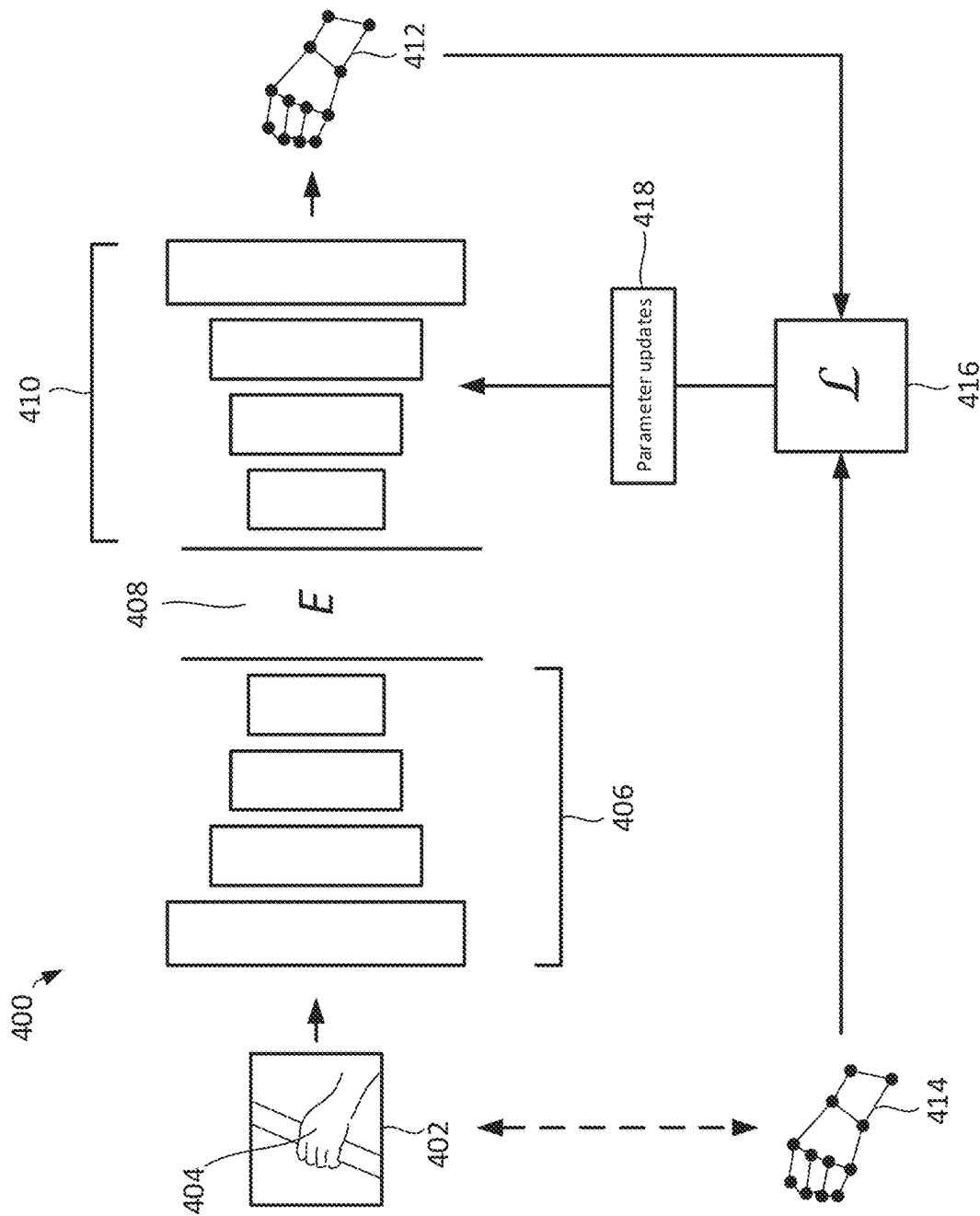


FIG. 4

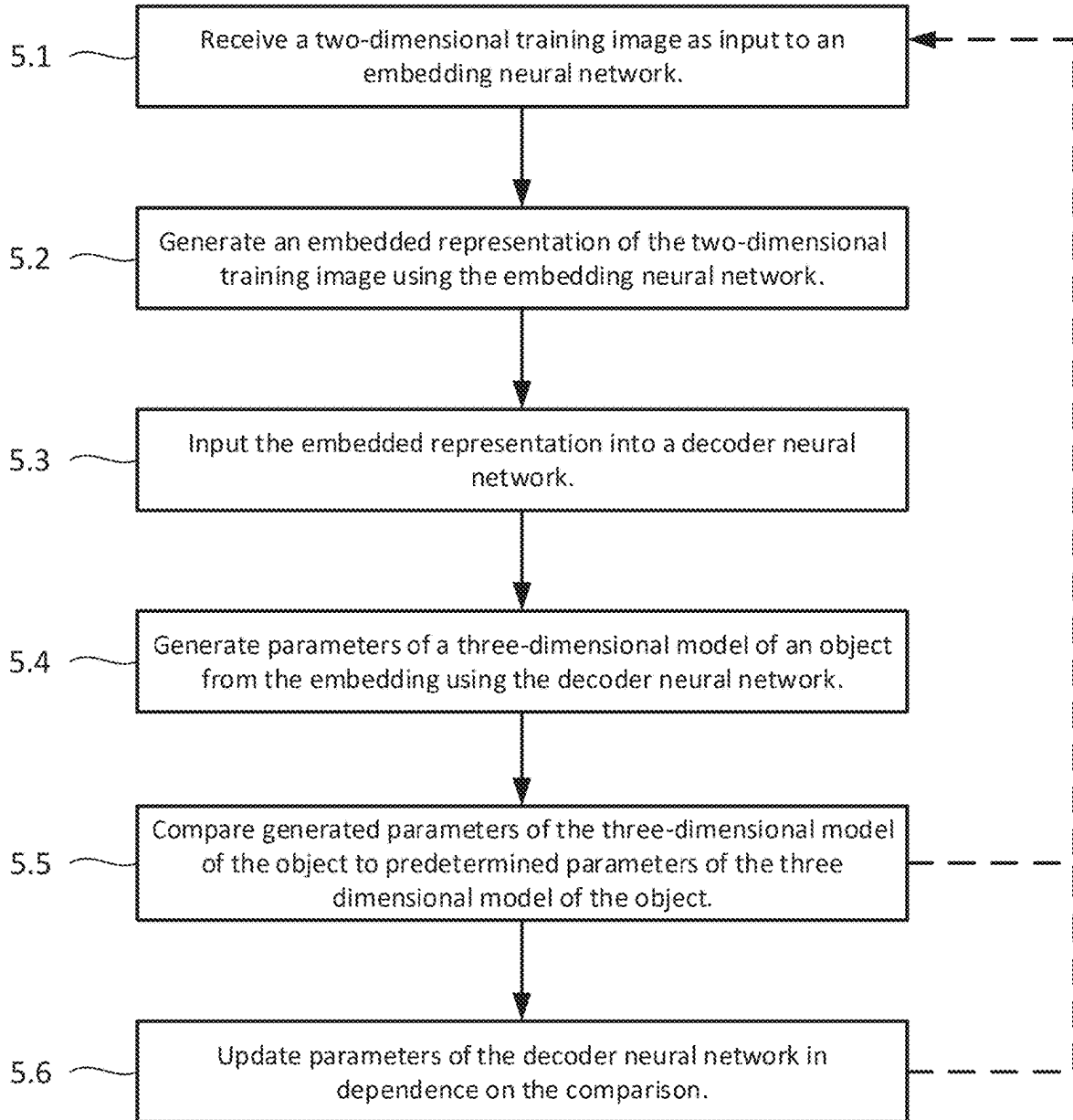


FIG. 5

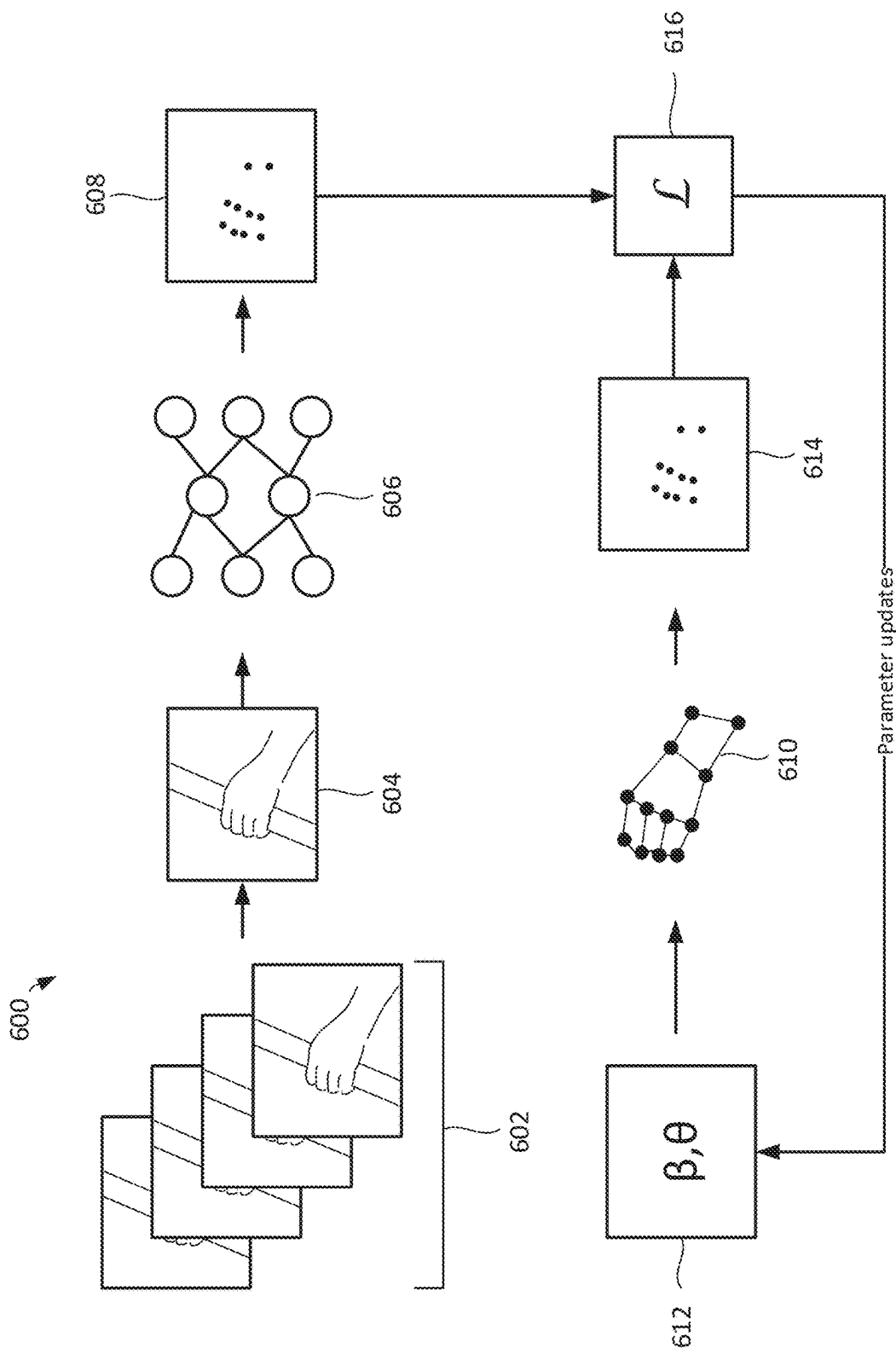


FIG. 6

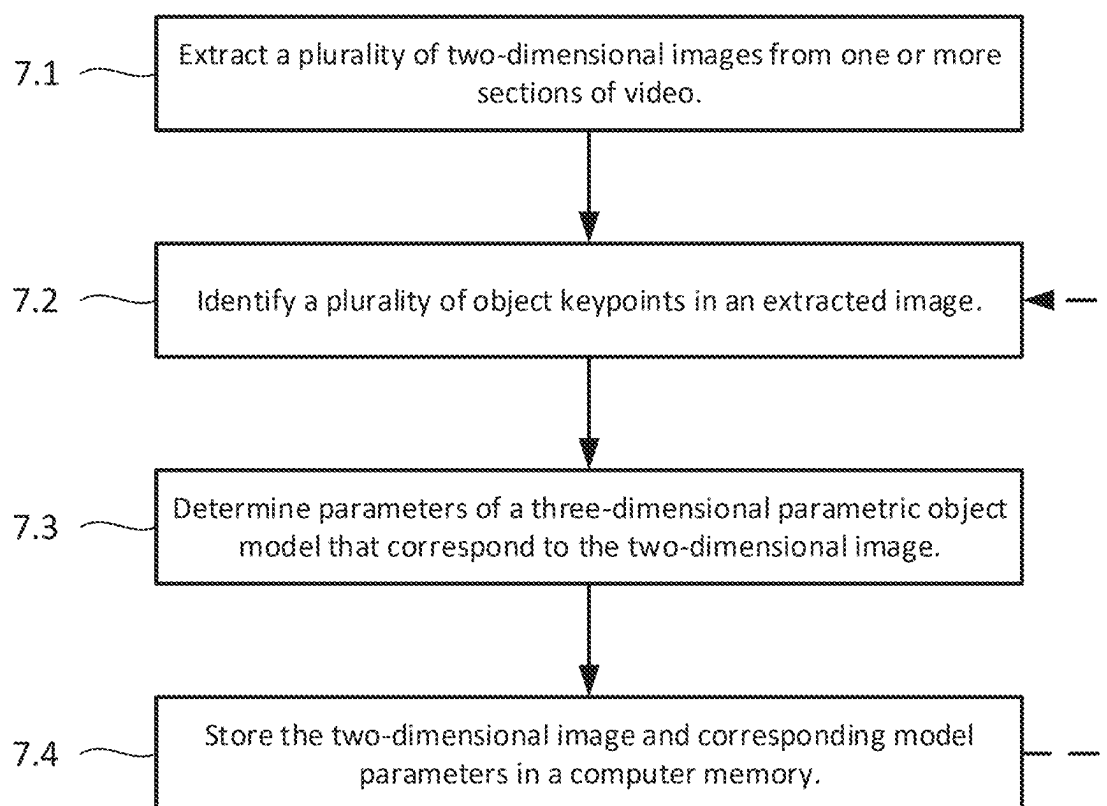


FIG. 7

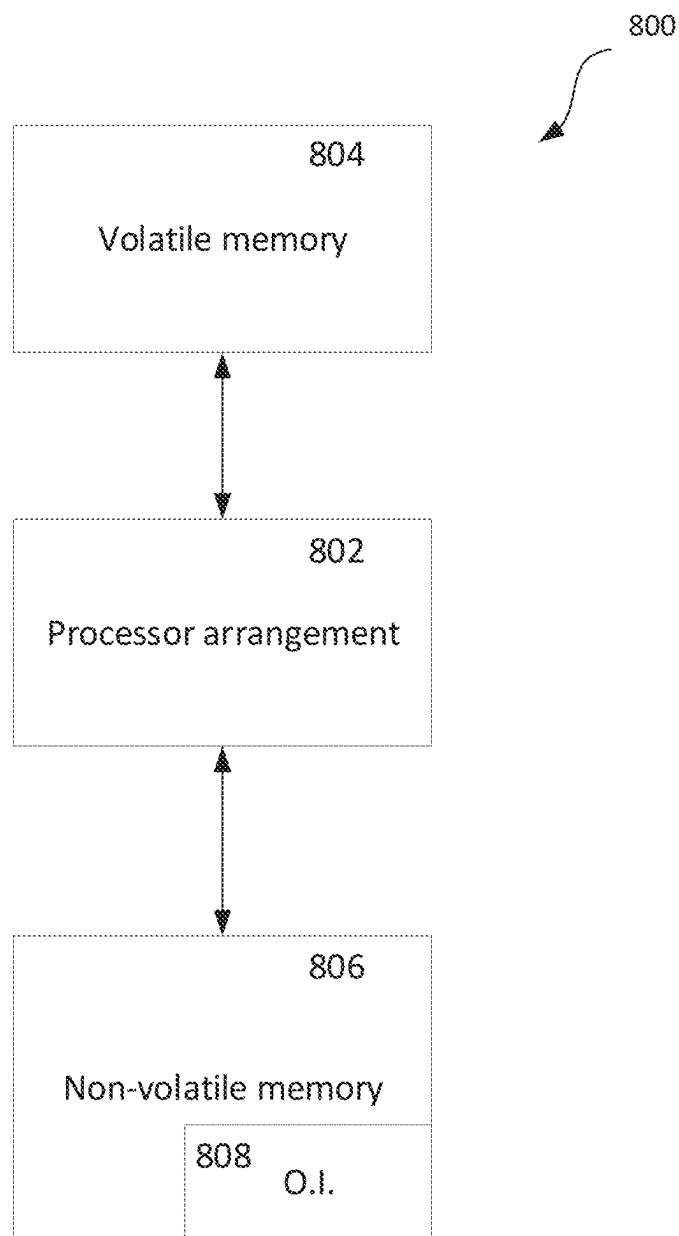


FIG. 8

GENERATING THREE-DIMENSIONAL OBJECT MODELS FROM TWO-DIMENSIONAL IMAGES

CLAIM OF PRIORITY

[0001] This application is a continuation of U.S. patent application Ser. No. 17/760,424, filed Aug. 9, 2022, which is a U.S. national-phase application filed under 35 U.S.C. § 371 from International Application Serial No. PCT/GB2020/050371, filed Feb. 17, 2020, and published as WO 2021/165628 on Aug. 26, 2021, each of which is incorporated herein by reference in its entirety.

FIELD

[0002] This specification discloses methods and systems for generating three-dimensional models of deformable objects from two-dimensional images.

BACKGROUND

[0003] Monocular three-dimensional (3D) reconstruction of objects can facilitate a broad range of applications in human-computer interaction, augmented reality, virtual telepresence, etc. In some applications, 3D reconstructions of hands may be particularly useful, such as in automated sign language recognition/generation and augmented reality control/interactions.

[0004] Current state-of-the-art methods are limited in that they do not easily generalise to samples captured in a non-laboratory environment (e.g. “in the wild”), and are often based on estimating locations of a sparse set of keypoints (e.g. hand joints and fingertip locations when generating 3D models of hands).

SUMMARY

[0005] According to one aspect of this disclosure, there is described a computer implemented method for generating a three dimensional model of deformable object from a two-dimensional image. The method comprises: receiving, as input to an embedding neural network, the two-dimensional image, wherein the two dimensional image comprises an image of an object; generating, using the embedding neural network, an embedded representation of a two-dimensional image; inputting the embedded representation into a learned decoder model; and generating, using the learned decoder model, parameters of the three dimensional model of the object from the embedded representation.

[0006] According to a further aspect of this disclosure, there is described a computer implemented method for training a learned decoder model to generate a three dimensional model of a deformable object from a two-dimensional image. The method comprises: receiving, as input to an embedding neural network, a two-dimensional training image, wherein the two dimensional training image comprises an object and is associated with predetermined parameters of a three-dimensional model of the deformable object; generating, using the embedding neural network, an embedded representation of a two-dimensional training image; inputting the embedded representation into a decoder model; generating, using the decoder model, generated parameters of the three dimensional model of the object from the embedded representation, comparing, using a loss function, generated parameters of the three dimensional model of the object to the predetermined parameters of the three dimensional

model of the object associated with the two-dimensional training image; updating parameters of the decoder model in dependence on the comparison.

[0007] The parameters of the three dimensional model of the object may comprise positions of a plurality of nodes in a mesh, and wherein the loss function comprises a vertex term comparing generated positions of nodes to corresponding predetermined positions of nodes associated with the two-dimensional training image. The loss function may comprise an edge term comparing distances between generated positions of nodes to distances between corresponding predetermined positions of nodes associated with the two-dimensional training image. The predetermined parameters of a three dimensional model of the object associated with the two-dimensional training image may be derived by fitting a three dimensional object model to the two training dimensional image.

[0008] The above methods may each be combined with one or more of the following features, either alone or in combination.

[0009] The embedding neural network may comprise a residual neural network.

[0010] The three-dimensional model may comprise a mesh model and the parameters of the three dimensional model of the object may comprise mesh parameters. The decoder model may comprise a decoder neural network.

[0011] The decoder model may comprise a decoder neural network comprising one or more two-dimensional convolutional layers, and wherein parameters of the three dimensional model of a hand comprise three-dimensional co-ordinates of nodes of a mesh. The decoder neural network may comprise one or more upscaling layers.

[0012] The initial layers of the decoder neural network may comprise a fully connected layer and a reshaping layer.

[0013] The decoder model may comprise a linear layer and wherein parameters of the three dimensional model of a hand comprise three-dimensional co-ordinates of nodes of a mesh.

[0014] The deformable object may be a hand.

[0015] According to a further aspect of this disclosure, there is described a computer implemented method of generating a training dataset for training a machine learning model to generate a three-dimensional model of a deformable objects from a two-dimensional image. The method comprises: extracting a plurality of two-dimensional images from one or more sections of video data, the two dimensional image each comprising one or more deformable objects; for each of the two-dimensional images in the plurality of two-dimensional images: identifying, using a keypoint identification model, a plurality of object keypoints in the two-dimensional image; and determining parameters of a three-dimensional parametric model of an object, the parameters corresponding to the deformable object in the two-dimensional image, wherein determining the parameters comprises fitting a three-dimensional parametric model of the object to the two-dimensional image using the identified object keypoints and corresponding object keypoints in the three-dimensional parametric model; and storing, in a computer memory, the two-dimensional image and the corresponding parameters of the three-dimensional parametric model.

[0016] The one or more sections of video data may comprise one or more sections of video of use of sign language.

[0017] Fitting the three-dimensional parametric model of the object to the two-dimensional image may comprise applying an iterative optimisation procedure to an objective function, wherein the objective function comprises a reprojection term comparing locations of object keypoints identified in the two dimensional image with locations of corresponding object keypoints in a two-dimensional projection of the three-dimensional parametric model. The reprojection term may comprise a mask that applies a weighting to the corresponding object keypoints in the two-dimensional projection. The objective function may further comprise a length term comparing distances between object keypoints identified in the two dimensional image and distances between corresponding object keypoints in a two-dimensional projection of the three-dimensional parametric model. The objective function may further comprise a regularisation term penalising deviations from a predefined mean object pose.

[0018] The three-dimensional parametric model of the object may be a mesh model parametrised by parameters comprising one or more shape parameters and/or one or more pose parameters. The parameters of the three-dimensional parametric model of the object may further comprise one or more camera parameters. The corresponding object keypoints in the three-dimensional parametric model may be identified from the mesh model using a sparse regression model.

[0019] The deformable object may be a hand. The one or more sections of video data may comprise a plurality of sections of video of use of sign language, wherein the plurality of sections of video comprise the use of sign language from different sign languages. The keypoint extraction model may be a two-dimensional pose estimation model configured to identify joint positions and/or fingertip positions of hands in the two-dimensional images.

[0020] According to a further aspect of this disclosure, there is described a computer implemented method of training a machine learned model for generating a three dimensional model of an object, the method comprising the use of a training dataset generated according to the method described above.

[0021] According to a further aspect of this disclosure, there is described a computer implemented method for generating a three dimensional model of an object, the method comprising the use of a machine learned model trained using any of the training methods disclosed herein.

[0022] According to a further aspect of this disclosure, there is described apparatus comprising a processor and a memory, the memory comprising computer readable instructions that, when executed by the processor, cause the apparatus to perform any one or more of the methods disclosed herein.

[0023] According to a further aspect of this disclosure, there is described a computer program product comprising computer readable code that, when executed by a computer, causes the computer to perform any one or more of the methods disclosed herein.

BRIEF DESCRIPTION OF THE DRAWINGS

[0024] Embodiments will now be described by way of non-limiting examples with reference to the accompanying drawings, in which:

[0025] FIG. 1 shows an overview of an example method of generating a three dimensional model of an object from a two-dimensional image;

[0026] FIG. 2 shows a flow diagram of an example method of generating a three dimensional model of an object from a two-dimensional image;

[0027] FIG. 3 shows an example of a structure of a decoder neural network;

[0028] FIG. 4 shows an overview of an example method for training a model to generate a three dimensional model of an object from a two-dimensional image;

[0029] FIG. 5 shows a flow diagram of an example method for training a model to generate a three dimensional model of an object from a two-dimensional image;

[0030] FIG. 6 shows an overview of an example method for generating a training dataset for training a machine learning model to generate a three-dimensional model of an object from a two-dimensional image;

[0031] FIG. 7 shows a flow diagram of an example method for generating a training dataset for training a machine learning model to generate a three-dimensional model of an object from a two-dimensional image; and

[0032] FIG. 8 shows an example of a system/apparatus for performing the methods disclosed herein.

DETAILED DESCRIPTION

[0033] Example implementations provide system(s) and methods for monocular three-dimensional object model reconstruction.

[0034] Methods/systems for generating three-dimensional object models from two-dimensional images are introduced that use a neural network architecture comprising an image encoder followed by a decoder model. The methods/systems may be trained based on a direct three-dimensional object mesh reconstruction loss. A training dataset for use in the training may be generated automatically from videos containing example images of objects of a given type (e.g. hands). While the methods and systems used herein can be used to generate models for a wide range of object types, they are particularly suited to generating three-dimensional models of hands in images.

[0035] The methods described herein can result in a lower pose estimation error for three-dimensional object models generated from in-the-wild images when compared to current methods. The methods described herein can also outperform previous methods of object pose estimation, particularly hand pose estimation, without overfitting or tuning to a particular dataset. The methods described herein thus provide robust and accurate means of determining three-dimensional object models from two-dimensional images, from which accurate estimations of object poses can be determined. Accurate object pose information, particularly hand pose estimation, has many applications in human-computer interaction, such as automatic sign language recognition, augmented/virtual reality control etc.

[0036] Furthermore, the methods of dataset generation described herein can produce a labelled dataset of object images from raw in-the-wild hand images/video without the need for manual labelling.

[0037] In the following, the methods and systems of generating three-dimensional object models from two-dimensional images are mainly described in relation to the object being a hand, i.e. generating a 3D model of a hand from a 2D image containing a hand. However, it will be

appreciated that the methods can alternatively or additionally be applied to other objects. Examples include, but are not limited to: faces; human/animal bodies and/or parts thereof; vehicle, such as cars or planes for example; and/or robots and/or robot parts. The methods and systems can be used with objects for which deformable models exist, i.e. deformable objects. A deformable object may be considered to be an object/object category whose surface can be parametrised using a mesh with a fixed/predetermined topology.

[0038] As used herein, the term “hand” is preferably used to connote a human hand. However, it may alternatively or additionally be used to connote one or more of: an animal claw/paw/hand; a robotic hand or gripper; and/or appendages of fictional creatures.

[0039] FIG. 1 shows an overview of an example method of generating a three dimensional model of an object from a two-dimensional image. In this example, the object is a hand, though may alternatively be another type of deformable object. The method **100** takes as input a 2D image **102** comprising one or more hands **104**. An encoder model **106**, such as an encoder neural network, processes the 2D image **102** to generate an embedding **108**, E, of the image. The embedding **108** is input into a decoder model **110** (in this example, a decoder neural network) that processes the embedding **108** to generate a 3D model **112** of one or more of the hands **104** in the 2D image **102**.

[0040] The two-dimensional image **102**, I, comprises a set of pixel values corresponding to a two-dimensional array. For example, in a colour image, $I \in \mathbb{R}^{H \times W \times 3}$, where H is the height of the image in pixels, W is the width of the image in pixels and the image has three colour channels (e.g. RGB or CIELAB). The two dimensional image may, in some embodiments, be in black-and-white/greyscale.

[0041] The encoder neural network **106** is a model for generating an embedding (e.g. a latent representation/vector) of an input image. The encoder neural network **106** may be a neural network pre-trained for image embedding. Such a neural network may include one or more convolutional layers. The neural network may include one or more skip connections. The resulting embedding **108** may, for example, comprise a distribution over a set of image features. An example of such a neural network is a ResNet neural network, such as the ResNet-50 network. Other examples include, but are not limited to, HRNet (see “Deep High-Resolution Representation Learning for Human Pose Estimation”, K. Sun et al., CVPR 2019), MobileNetV2 (see “MobileNetV2: Inverted Residuals and Linear Bottlenecks”, M. Sandler et al., CVPR 2019), and EfficientNet (see “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks”, M. Tan et al., ICML 2019).

[0042] The decoder model **110** takes the image embedding **108** as input and processes it to generate the output three-dimensional model **112** of the hand **104** in the input image. In embodiments where the decoder model **110** is a neural network, the image embedding is input into an input layer of the decoder neural network. The embedding **108** is then processed through a sequence of decoder neural network layers to generate the output three-dimensional model **112** of the hand **104** in the input image.

[0043] The decoder model **110** is a learned model (i.e. a trained model) that has been trained on a set of training data to generate the output three-dimensional model **112** of the

hand **104** in the input image from an embedding of that image. Examples of training such a model are described below.

[0044] The decoder model **110** may comprise one or more spatial convolutional layers. The spatial convolutions of these layers may operate in pixel-space. The decoder model **110** may comprise one or more fully connected layers. For example, the initial layer may be a fully connected layer. Alternatively or additionally, the decoder model may comprise one or more upscaling layers for increasing the dimensionality of their input. In some embodiments the decoder model comprises or consists of a single linear layer.

[0045] Examples of structures of decoder neural networks/models are described in further detail below in relation to FIG. 3. In general, a neural network comprises a plurality of layers of nodes, each node associated with one or more parameters. The parameters of each node of the neural network may comprise one or more weights and/or biases. The nodes take as input one or more outputs of nodes in the previous layer. The one or more outputs of nodes in the previous layer are used by the node to generate an activation value using an activation function and the parameters of the neural network. The activation functions may comprise the ReLU activation function, though other activation functions (such as a sigmoid or tanh activation function) may alternatively or additionally be used.

[0046] The output three-dimensional model **112** comprises a three-dimensional representation of the hand **104** in the input image **102**. In general, any parameters that define the three-dimensional model of a hand corresponding to the hand **104** in the input image **102** may be output by the decoder neural network **112**. For example, the three-dimensional model **112** may be a mesh model, defined by a set of mesh parameters. The mesh parameters may comprise: vertex/node locations; edges defined between pairs of vertices/nodes in the mesh; and/or faces (i.e. a closed set of edges, such as a triangle face or quad face). An example of such a mesh model for a hand is the MANO model, though other mesh models may alternatively be used. In some embodiments, the output of the decoder neural network **112** may simply be locations of mesh vertices/nodes in a pre-defined model rather than the full mesh itself.

[0047] FIG. 2 shows flow diagram of an example method of generating a three-dimensional model of an object from a two-dimensional image. The method may be implemented on a computer. The object may be a hand, as described above in relation to FIG. 1.

[0048] At operation **2.1**, a two-dimensional image is received as input to an embedding neural network. The two dimensional image comprises one or more objects of a given type, e.g. one or more hands.

[0049] At operation **2.2**, an embedded representation of the two-dimensional image is generated using the embedding neural network. The embedding neural network may comprise a residual neural network, i.e. a neural network with one or more skip connections. The embedding neural network may comprise one or more convolutional layers that each applies one or more convolutional filters to the input image and/or output of a previous layer in the embedding neural network.

[0050] At operation **2.3**, the embedded representation is input into a decoder model. The decoder model may be a decoder neural network. The initial layers (e.g. one of the input layers or a layer following the input layer) of the

decoder neural network may comprise a fully connected layer. The fully connected layer may be the input layer of the decoder neural network that takes the embedded representation as input and produces a feature map from it. The feature map may be defined in a pixel-like space (e.g. one or more arrays of pixel values) or in a mesh space (e.g. values are associated with mesh nodes). A reshaping layer may follow the fully connected layer to reshape the feature maps for use by subsequent layers.

[0051] At operation 2.4, parameters of a three dimensional model of a hand are generated from the embedding of the two-dimensional image using the decoder model. The three-dimensional model of the hand may be a mesh model, with the parameters being mesh parameters. For example, the mesh parameters may comprise: locations of nodes in the mesh; edges connecting pairs of nodes in the mesh; and/or mesh shapes (such as mesh triangles or mesh quadrilaterals, for example).

[0052] In some embodiments, the decoder model may comprise or consist of a single linear layer. The single linear layer can provide direct object regression to the three-dimensional model. The linear layer may comprise a matrix with learned values for the matrix elements that takes as input a vector obtained by reshaping the image embedding output from the embedding neural network.

[0053] In some embodiments, the decoder model may comprise a convolutional neural network, comprising one or more convolutional layers. The convolutional neural network may further comprise one or more upscaling layers configured to increase the dimensionality of an output of a convolutional layer.

[0054] FIG. 3 shows an example of a decoder neural network structure. The decoder neural network 300 is configured to generate a 3D mesh model of a hand from an embedding of a 2D image of a hand.

[0055] The decoder neural network may comprise one or more two-dimensional convolutional layers 302, i.e. layers that each configured to apply one or more two-dimensional convolutional filters to an array of, for example, pixel values and/or feature maps. In embodiments where such two-dimensional convolutional layers are used, the output of the decoder neural network may be a set of node/vertex coordinates for a mesh.

[0056] The decoder neural network may comprise one or more upscaling layers 304. The upscaling layers 304 are configured to increase the dimensionality of their input by a predetermined factor. In the example shown, the upscaling layers 304 upscale their input by a factor of two, though other upscaling factors may alternatively be used. The upscaling layers 304 may be deconvolutional layers.

[0057] The initial layers (e.g. one of the input layers or a layer following the input layer) of the decoder neural network may comprise a fully connected layer 306. The fully connected layer 306 may be the input layer of the decoder neural network that takes the embedded representation as input and produces a feature map from it. The feature map may be defined in a pixel-like space (e.g. one or more arrays of pixel values) or in a mesh space (e.g. values are associated with mesh nodes). In the example shown, the fully connected layer 306 outputs a (51×48)-dimensional vector.

[0058] A reshaping layer 308 may follow the fully connected layer to reshape the feature maps for use by subsequent layers. The reshaping layer 308 may convert a vector

output by the fully connected layer 306 into a tensor. In the example shown, the reshaping layer 308 converts the (51×48)-dimensional vector output by the fully connected layer 306 into a 51-by-48 dimensional matrix. Other reshaping are possible, depending on the nature of the subsequent layers of the neural network 300.

[0059] In some embodiments, the output of the decoder neural network comprises or consists of three-dimensional co-ordinates of nodes of a mesh.

[0060] In some embodiments, the decoder model may be a linear regression model. The problem may be re-interpreted to one of direct regression from an image path in the 2D image to the locations of mesh vertices. In these embodiments, an embedding neural network is used as described above, with a single layer operating on the top layer of the embedding neural network block. The resulting tensor is reshaped into a D-dimensional vector, then multiplied by a matrix of dimension (N*3)×D, where N is the number of vertices in the mesh, to give the three-dimensional coordinates of the N mesh vertices. The values of the matrix elements used in the matrix may be learned, for example, using the methods described below in relation to FIGS. 5 and 6. In some embodiments, an (N*3) vector of learned parameters may also be added to the output of the matrix multiplication.

[0061] As an example, the embedding network may be a ResNet-50 network. The output 5×5×512 tensor representing the embedding is reshaped to a D=12800 dimensional vector. The reshaped vector is multiplied by a (722*3)×12800 matrix to deliver the 3D coordinates of 722 mesh vertices.

[0062] Embodiments based on the linear regression model are particularly suited to lightweight implementations on mobile devices. They do not use any specialised convolutions, such as geometric convolutions, and do not use deconvolutional based filters. Instead, they rely on generic convolutional network layers. The models can also be extended to apply to the entirety of an image in a fully convolutional manner, rather than processing individual patches.

[0063] FIG. 4 shows an overview of a method 400 for training a model to generate a three-dimensional model of an object 412 from a two-dimensional image 402. In the example shown, the object 412 is a hand, though the model may alternatively be trained to generate a three-dimensional model of other object types.

[0064] Training image 402 comprising an object (in this case a hand) 404 is selected from a training dataset of images containing hands. Each training image 402 in the training dataset is associated with a predetermined (i.e. known) three-dimensional model 414 of the object 404 in the image. The training dataset may, for example, be generated using the method described below in relation to FIGS. 6 and 7. Alternatively, the training dataset may be generated manually. In general, any method of generating a dataset of images of objects with corresponding three-dimensional models may be used.

[0065] The predetermined three-dimensional model 414 of the hand may be stored as a full mesh model of a hand associated with a corresponding two-dimensional image. Alternatively, the predetermined three-dimensional model 414 of the hand may be generated from stored parameters of

a parametric model that are associated with the corresponding two-dimensional image (e.g. parameters of the MANO model).

[0066] The selected image **404** is input into an embedding neural network **406**. The embedding neural network **406** generates an embedding **408** of the input image **402**, i.e. a latent representation of the image.

[0067] The embedding **408** is input into a decoder neural network **410** that generates a three-dimensional model of a hand from the embedding. The decoder neural network **410** may, for example, be a convolutional neural network, as described above. The three-dimensional model **412** of a hand may be a mesh model (i.e. parameters of a mesh representing the hand, such as vertex/node locations, edges, and or mesh polygons).

[0068] The generated three-dimensional model of the hand **412** is compared to the predetermined three-dimensional model of the hand **414** associated with the input image **402**. The comparison may be performed by means of a loss function **416**, $\mathcal{L}$, that compares properties of the generated three-dimensional model of the hand **412** to properties of the predetermined three-dimensional model of the hand **414** associated with the input image **402**.

[0069] The loss function **416** may comprise a vertex term, $\mathcal{L}_{\text{vertex}}$. The vertex term compares locations of vertices/nodes in the generated three-dimensional model of the object **412** to locations of vertices/nodes in the predetermined three-dimensional model of the object **414** associated with the input image **402**. The vertex term may depend on differences between locations of vertices/nodes in the generated three-dimensional model of the object **412** and corresponding locations of vertices/nodes in the predetermined three-dimensional model of the object **414** associated with the input image **402**. An example of such a vertex loss function term is given by:

$$\mathcal{L}_{\text{vert}} = \lambda_{\text{vertex}} \|\hat{\mathbf{Y}} - \mathbf{Y}\|_1$$

where $\hat{\mathbf{Y}}$ is the ground-truth mesh vertex locations (i.e. the locations of vertices/nodes in the predetermined three-dimensional model of the object **414** associated with the input image **502**), $\mathbf{Y}$ is the generated mesh vertex locations (i.e. the locations of vertices/nodes in the generated three-dimensional model of the object **412** generated from the same input image **402**), and λ_{vertex} is a hyperparameter controlling the relative importance of this term in the loss function **416** when the loss function comprises a plurality of terms. The hyperparameter λ_{vertex} may, for example, lie between 0.001 and 1, for example between 0.001 and 0.1, such as between 0.005 and 0.015, e.g. $\lambda_{\text{vertex}}=0.001$. In the above example, an L1-norm is used, though it will be appreciated that other norms may alternatively be used, such as an L2 norm.

[0070] The loss function **416** may alternatively or additionally comprise an edge term, $\mathcal{L}_{\text{edge}}$. The edge term may alternatively be referred to as an “edge length preservation” component. The edge term compares lengths of edges between nodes in the generated three-dimensional model of the object **412** to lengths of edges between nodes in the predetermined three-dimensional model of the object **414** associated with the input image **502**. The edge term may depend on differences between lengths of edges in the generated three-dimensional model of the object **412** and

corresponding lengths of edges in the predetermined three-dimensional model of the object **414** associated with the input image **502**. An example of such an edge loss function is given by:

$$\mathcal{L}_{\text{edge}} = \lambda_{\text{edge}} \sum_{(u,v) \in \epsilon_{\text{mesh}}} \left\| (\hat{\mathbf{Y}}_v - \hat{\mathbf{Y}}_u) - (\mathbf{Y}_v - \mathbf{Y}_u) \right\|^2$$

where ϵ_{mesh} is the set of mesh edges (e.g. a pair of nodes (u, v) defining the end points of the edge), $\hat{\mathbf{Y}}_v$ and $\hat{\mathbf{Y}}_u$ are the nodes corresponding to the endpoints of the edge (u, v) in the ground-truth mesh **514**, and $\mathbf{Y}_v$ and $\mathbf{Y}_u$ are the nodes corresponding to the endpoints of the edge (u, v) in the generated mesh **412**. λ_{edge} is a hyperparameter controlling the relative importance of this term in the loss function **416** when the loss function comprises a plurality of terms. The hyperparameter λ_{edge} may, for example, lie between 0.001 and 1, for example between 0.001 and 0.1, such as between 0.005 and 0.015, e.g. $\lambda_{\text{edge}}=0.001$. In the above example, an L2-norm is used, though it will be appreciated that other norms may alternatively be used, such as an L1 norm.

[0071] A combined loss function may be given by the sum of the vertex term and the edge term, giving:

$$\mathcal{L} = \lambda_{\text{vertex}} \|\hat{\mathbf{Y}} - \mathbf{Y}\|_1 + \lambda_{\text{edge}} \sum_{(u,v) \in \epsilon_{\text{mesh}}} \left\| (\hat{\mathbf{Y}}_v - \hat{\mathbf{Y}}_u) - (\mathbf{Y}_v - \mathbf{Y}_u) \right\|^2$$

[0072] In some implementations, $\lambda_{\text{edge}} = \lambda_{\text{vertex}}$. Additional terms may be included in the loss function, such as a pose estimation loss. However, it has been found that the inclusion of a pose estimation loss has limited effect on the training results, so can be omitted in some embodiments.

[0073] The loss function **416** is used to determine parameter updates **418** for the decoder model **410**. For example, weights and/or biases of the decoder neural network **418** are updated based on the comparison of the generated three-dimensional model of the object **412** to the known three-dimensional model of the object **414**. For example, an optimisation procedure may be applied to the loss function **416** in order to determine the parameter updates **418**. Examples of such optimisation procedures include, but are not limited to, gradient descent methods, such as stochastic gradient descent, simulated annealing, momentum methods, such as Adam, and the like.

[0074] In some implementations, parameters of the encoder neural network **406** may also be updated based on the loss function **416** using any of the training methods described herein. Alternatively, the parameters of the encoder neural network **406** may be fixed during training of the decoder model **412**.

[0075] In some implementations, parameter updates **418** are determined once a batch of training images **402** has been processed. An average/sum of the loss function **416** over the batch of training images may be used to determine the parameter updates **418**. The batch of training images comprises a plurality of different training images **402** from the training dataset. The batch size (i.e. the number of distinct training images in a batch) may, for example be between 10 and 100 images, preferably between 25 and 50 images, such as 32 images.

[0076] The training may be iterated until a threshold condition is satisfied. The threshold condition may one or more of: a threshold number of iterations; a threshold number of training epochs (i.e. forward and backward passes through the training dataset); a threshold accuracy being obtained on a test dataset comprising two-dimensional images of hands with known three-dimensional hand models; the loss function being optimised to within a threshold value; and/or some other convergence criterion or criteria. Where a threshold number of epochs are used, the threshold number may, for example, be between 100 and 200 epochs, preferably between 125 and 175 epochs, such as 150 epochs.

[0077] The parameter updates 418 may determine based on a learning rate of the optimisation procedure used, e.g. a “step size” in gradient descent methods. The learning rate may be constant throughout the training. Alternatively, the learning rate may vary throughout the training, for example by decaying from an initial learning rate by a predetermined factor at pre-defined training epochs. As an example, the initial learning rate using an Adam optimiser may be 10^{-4} , and decay by a factor of 0.1 at the 90-th and 120-th epochs. Other learning rate decay routines may alternatively be used.

[0078] In some implementations, the input image and/or three-dimensional models may undergo pre-processing and/or post-processing. For example, the input images 402 may be augmented with randomly crops and/or random transformations (e.g. translations, rotations, scaling etc.). This can enhance the performance of the trained model on real-world examples, where input images can’t be cropped based on ground truth annotations. Alternatively or additionally, the input images 502 may be normalised with the mean and standard deviation from a known image dataset, such as ImageNet. Output meshes may 412 may be normalised based on statistics calculated on a subset of the known meshes 414.

[0079] FIG. 5 shows a flow diagram of an example method for training a neural network to generate a three-dimensional model of an object of a given type from a two-dimensional image. The method may be implemented on a computer. The object type may be a hand.

[0080] At operation 5.1, a two-dimensional training image is received as input to an embedding neural network. The two-dimensional training image comprises an image of an object of a given type, and is associated with a corresponding predetermined three-dimensional model of the object in the image. The predetermined three-dimensional object model may have been derived by fitting a three dimensional object model to the two training dimensional image.

[0081] At operation 5.2, an embedding of the two-dimensional training image is generated using the embedding neural network, as described above in relation to FIGS. 1 and 2. The embedding neural network may be a residual neural network.

[0082] At operation 5.3, the embedded representation is input into a decoder model. The decoder model comprises a parameterised model, such as a decoder neural network or linearised regression model, which can be trained using machine learning techniques.

[0083] At operation 5.4, a three dimensional model (e.g. parameters of a three-dimensional model) of the hand in the two-dimensional image is generated using the decoder model, as described above in relation to FIGS. 1-3.

[0084] At operation 5.5, the generated three-dimensional model is compared to the predetermined three-dimensional

model of the object (i.e. parameters of the generated three-dimensional model are compared to parameters of the predetermined three-dimensional model). The comparison is performed using a loss function (which may also be referred to as an objective function).

[0085] The loss function may comprise a vertex term comparing generated positions of nodes/mesh vertices to corresponding predetermined positions of nodes/mesh vertices in the predetermined model associated with the two-dimensional training image. The loss function may alternatively or additionally comprise an edge term comparing distances between generated positions of nodes/mesh vertices to distances between corresponding predetermined positions of nodes/mesh vertices in the predetermined three-dimensional model associated with the two-dimensional training image.

[0086] At operation 5.6, parameters of the decoder model are updated in dependence on the comparison. The parameters may be updated using an optimisation routine, such as stochastic gradient descent, etc. Operations 5.1-5.5 may be iterated a plurality of times, each with a different training image from a training batch. The parameters of the decoder model updated based on an expectation value of the loss/objective function taken over the training images of a batch.

[0087] Operations 5.1 to 5.6 may be iterated until a threshold condition is satisfied. The threshold condition may be a threshold number of training epochs. The threshold condition may comprise one or more convergence criteria defined, for example, on a set of validation images with corresponding predetermined three-dimensional models.

[0088] FIG. 6 shows an overview of an example method for generating a training dataset for training a machine learning model to generate a three-dimensional model of a hand from a two-dimensional image. The method may be performed by a computer. A training set generated using the methods described in relation to FIGS. 6 and 7 may be used as a training dataset/validation dataset in the methods described in relation to FIGS. 4 and 5.

[0089] Training models for three-dimensional object reconstruction may often be limited by the availability of labelled images that can be used in the training. Labelled images are often manually annotated, and/or captured in laboratory conditions, and thus may not capture the full range of object positions/poses etc. that occur in-the-wild.

[0090] Video data can provide multiple images of objects of a particular type. For example, when the objects of interest are hands, videos of sign language use may provide two-dimensional hand images in a wide range of realistic poses. Such videos may be obtained from publically available video repositories, such as internet video sites.

[0091] Automatically fitting three-dimensional object models to two-dimensional images extracted from such video data can result in a training dataset that more accurately reflects in-the-wild object poses. Training a three-dimensional object generation model on such training data can result in a more accurate model generation model when compared to manually labelled images and/or images captured in lab conditions.

[0092] The method 600 comprises receiving a section of video data 602, the video data comprising a plurality of frames, and extracting one or more two-dimensional images 604 comprising an object of a given type (e.g. in the example shown, the object type is hands). The video data 602 may be taken from a publically available source of video data, such

as a video hosting website. This can help ensure that the video data **602** represents “real-world” examples of images of the object type. In embodiments where the object type is a hand, the video data may comprise one or more segments of video data depicting the use of a sign language.

[0093] Each extracted image is fed into a keypoint identification model **606** that identifies a plurality of object keypoints **608** in the input image. The keypoint extraction model **606** may, for example, be a pre-trained neural network. The pre-trained neural network may comprise one or more convolutional layers. An example of such a model is OpenPose, though other models trained to identify keypoints of a given object type in an image may alternatively be used.

[0094] Locations, Y , of the identified keypoints **608** are used to iteratively fit a three-dimensional deformable model (i.e. a parametric model) of the object to the image. For example, a deformable model of a hand (such as the MANO model) may be iteratively fitted to images of hands.

[0095] The deformable model predicts N mesh vertices on the surface of the object through a differentiable function, $M(\beta, \theta)$, that maps shape, β , and pose, θ , parameters **612** to an instance of the model represented by an N -vertex mesh. In some embodiments, the deformable model may also predict locations a plurality of keypoints, K , of the model. In embodiments where the object type is hands, the keypoints may, for example, represent joints of the hand. The pose parameters, θ , may relate to angles of the joints in the hand. For example, in the MANO model, the pose parameters represent $K=16$ joints of the hand, and $\theta \in \mathbb{R}^{K \times 3}$.

[0096] The deformable model may additionally take as input one or more camera parameters. The camera parameters may, for example, comprise a scaling parameter, s , and a translation parameter, $\vec{T}_\delta \in \mathbb{R}^3$. Global orientation may be included in the pose parameters, θ . In such an example, the deformable model may be represented as:

$$M(\beta, \theta, \vec{T}_\delta, s; \phi): \mathbb{R}^{|\beta| \times |\theta| \times |\vec{T}_\delta| \times |s|} \rightarrow \mathbb{R}^{N \times 3}$$

where ϕ is a set of learned model parameters.

[0097] To prevent unrealistic object poses, such as impossible joint angles in a hand, the pose parameters may be constrained to lie on the convex hull of some pre-computed cluster centres. For example, where the pose parameters represent angles in the K joints of a hand, C Euler-angle clusters $P'_1, \dots, P'_C$ can be obtained via k-means for each joint, i . The pose variables for each joint can then be represented as

$$\theta_i = \frac{\sum_{c=1}^C \exp(w^c) P'_i{}^c}{\sum_c \exp(w^c)}.$$

[0098] This constrains the pose angles, θ_i , to take plausible values, while allowing optimisation over unconstrained variables w^c . The constrained angles may be represented in terms of a parameter matrix $w \in \mathbb{R}^{K \times \lambda C}$ while allowing global orientation to be unrestricted.

[0099] The three-dimensional model **610** may be initialised using an initial set of shape and pose parameters. The three-dimensional model may be used to determine one or more model keypoints **614** (i.e. keypoints of the model) corresponding to the keypoints **608** identified in the two-

dimensional image **604**. The model keypoints **614** may comprise the K keypoints predicted by the differentiable model M . The model keypoints may additionally or alternatively comprise one or more further keypoints, F . In a hand model, the K keypoints may comprise joints (e.g. $K=16$ in the MANO model), and the F further keypoints may comprise the five fingertips of a hand. A sparse regression model $J \in \mathbb{R}^{N \times (K+F)}$ may be used to regress the model keypoints **714** from the N -vertex mesh, delivering a hand pose, $J \in \mathbb{R}^{(K+F) \times 3}$.

$$J(\beta, \theta, \vec{T}_\delta, s) = J^T M(\beta, \theta, \vec{T}_\delta, s; \phi).$$

[0100] This model is iteratively fitted to the keypoints **608** identified in the two-dimensional image **604** by comparing the model keypoints **614** to the identified image keypoints **608**. The comparison may be performed using an objective function **614**. The fitting may be accomplished by applying an iterative optimisation procedure to the objective function **616**, such as gradient descent.

[0101] The objective function may comprise a 2D reprojection term, E_{2D} . The 2D reprojection term compares locations of the model keypoints **714** in a two-dimensional projection of the three dimensional model to the locations of the identified image keypoints **608**. A norm, such as a L2 or L1 norm, may be used as the objective function.

[0102] A mask, Λ_{2D} , may be applied to the projection of the three-dimensional model to reduce the influence of some keypoints and enhance the effects of others. For example, in a hand model the mask may reduce the influence of the metacarpophalangeal joints (MCP, i.e. the base of each finger), and enhance the influence of the fingertips and wrist. The MCP joints locations may be multiplied by a predetermined factor less than 1, such as between 0.2 and 0.9, or between 0.5 and 0.8, for example 0.7. The fingertips may be multiplied by a predetermined factor greater than 1, such as between 1.1 and 2.5, or 1.6 and 1.8, for example 1.7. The wrist may be multiplied by a predetermined factor greater than 1, such as between 1.5 and 3.5, or 2 and 3, for example 2.5.

[0103] An example of a 2D reprojection term may be represented symbolically as:

$$E_{2D}(\beta, w, \vec{T}_\delta, s) = \left\| \Lambda_{2D} \left(\Pi_K \left(J(\beta, P(w), \vec{T}_\delta, s) \right) \right) - Y \right\|^2$$

where Y is the locations of the identified image keypoints **608**, Π_k is an intrinsic camera projection to 2D and Λ_{2D} is a mask.

[0104] The objective function may further comprise a feature length term, E_f . In embodiments where the object type is a hand, this may be referred to as a “bone loss”. The feature length term ensures that the lengths of edges in the object skeleton tree, ϵ , are preserved. The feature length term compares the lengths of edges between keypoints in the 2D reprojection of the three dimensional model to the lengths of edges between identified keypoints **608** in the two-dimensional image **604**. A norm, such as a L2 or L1 norm, may be used as the objective function.

[0105] An example of a feature length term may be represented symbolically as:

$$E_f(\beta, w, \vec{T}_\delta, s) = \sum_{(i,j) \in \epsilon} \|(J_{2D_j} - J_{2D_i})^2 - (Y_j - Y_i)^2\|^2$$

where the sum is taken over pairs of points (i,j) in the object skeleton tree, ϵ , and $J_{2D_i} = \Pi_K (J(\beta, P(w), \vec{T}_\delta, S))_i$ is the location of feature i in the 2D projection of the three dimensional model.

[0106] In some embodiments, the objective function may further comprise a regularisation term, E_{reg} . The regularisation term penalises deviations from a mean object pose, and can help ensure realistic deformations. Assuming that the parameters θ and β are defined such that the mean object pose corresponds to $\theta=0$ and $\beta=0$, an example of a regularisation term is given by:

$$E_{reg}(\beta, \theta) = \lambda_\theta \|\theta\|^2 + \lambda_\beta \|\beta\|^2$$

where λ_β and λ_θ are experimentally chosen hyperparameters. In some embodiments, λ_θ may be less than 1, such as lying in the range 0.01 and 0.5, such as between 0.05 and 0.2, for example 0.1. In some embodiments, λ_β may be greater than 100, such as lying in the range 100 and 10000, such as between 500 and 1500, for example 1000.

[0107] The objective function may comprise one or more of the reprojection term, the feature length term and the regularisation term. The objective of the optimisation procedure may be to minimise the following objective:

$$\{\beta^*, w^*, \vec{T}_\delta^*, s^*\} = \arg \min_{\beta, w, \vec{T}_\delta, s} (E_{2D} + E_f + E_{reg}).$$

[0108] As an example, the Adam optimiser may be used to apply an optimisation procedure to the objective function. Different learning rates may be used for the camera, pose and shape parameters (for example, 10^{-2} , 10^{-2} and 10^{-5} respectively). The learning rate may decay by a predetermined factor after a predetermined number of iterations of the optimisation routine. For example, the learning rate may decay by a factor of 0.95 every 500 iterations. During some sequences of iterations, one or more of the parameter sets may be frozen. For example, in an embodiment where the object is a hand, a first number of iterations may apply the optimisation procedure over camera parameters and global orientation, with the joint set is reduced to a wrist and MCP joints excluding thumb. The first number of iterations may lie between 500 and 3000 iterations, such as between 1000 and 2000 iterations, for example 1500 iterations. Afterwards, the optimisation procedure may be applied over all parameters for a second number of iterations. The second number of iterations may lie between 500 and 4000 iterations, such as between 2000 and 3000 iterations, for example 2500 iterations.

[0109] As mentioned above, the data collection method may collect object images from a publically available video source (e.g. YouTube™). The method may, for example, iterate over a list of video links, download a video, extract

frames, run OpenPose, fit a three-dimensional model (e.g. MANO for hands) to each frame, and select a small subset of filtered samples. The depth of the projected mesh may be proportional to the ratio of standard deviations between X coordinates of the projected mesh and its world position. Sanity checks may be performed by thresholding total OpenPose confidence score, per-joint confidence score, and/or the mean squared error between projected model (e.g. MANO) joints and Open-Pose predictions normalized by the distance from the camera.

[0110] As an example, to create a training dataset, one-hundred-and-two videos are taken and at most 500 samples per video that meet the threshold conditions are randomly selected. In embodiments relating to hands, most of the samples may cover sign language conversations. These may be performed by people from a wide variety of nationalities and/or ethnicities. Some videos may include, for example, a hundred people signing a common phrase to a webcam all over the world. The validation and test sets may cover, for example, seven videos with an empty intersection of subjects with the training set. Test videos may be selected to be diverse and challenging, and may for example include conversations captured outdoor, dance poses, and sign languages from different regions (e.g. American, Australian, and Taiwanese sign languages). Additionally, the method may be performed on the COCO dataset, with, for example, 7,048 hand images extracted for training. A combined training set and validation and test sets may be created by combining the extracted COCO images with the images extracted from the videos.

[0111] FIG. 7 shows a flow diagram of an example method for generating a training dataset for training a machine learning model to generate a three-dimensional model of an object from a two-dimensional image.

[0112] At operation 7.1, a plurality of two-dimensional images is extracted from one or more sections of video data, the two dimensional image each comprising one or more objects of a given type, e.g. hands. The video data may be downloaded from an external repository, such as a publically available video streaming/sharing website. In embodiments where the object type is hands, the video may comprise one or more sections of video depicting the use of sign language.

[0113] At operation 7.2, for a given one of the extracted images, a plurality of object keypoints is identified in the two-dimensional image using a keypoint identification model. For example, the keypoint extraction model may be a two-dimensional pose estimation model configured to identify joint positions and/or fingertip positions of hands in the two-dimensional images. An example of such a model is OpenPose, though any keypoint extraction model that can identify object/hand keypoints may alternatively be used.

[0114] At operation 7.3, parameters of a three-dimensional parametric object model that correspond to the two-dimensional image are determined. Determining the parameters comprises fitting a three-dimensional parametric object model to the two-dimensional image using the identified object keypoints and corresponding object keypoints in the three-dimensional parametric object model.

[0115] The three-dimensional parametric hand model may be defined by a set of parameters comprising shape parameters, β , and pose parameters, θ . The parameters may also comprise scale and position parameters of a camera viewing the mesh. The model uses these parameters to generate a three-dimensional hand mesh via a differentiable function.

An example of such a model is the MANO model, though other parametric hand models may alternatively be used.

[0116] Keypoints of the three dimensional hand mesh may be identified using a sparse regression model. The keypoints may be projected onto the image plane and compared to the keypoints detected in the two-dimensional image, for example using a loss/objective function. The loss/objective function may comprise a re-projection term comparing locations of hand keypoints identified in the two dimensional image with locations of corresponding hand keypoints in a two-dimensional projection of the three-dimensional parametric model. A mask applying a weighting to the corresponding hand keypoints in the two-dimensional projection may be used in the re-projection term. The loss/objective function further comprises a bone length term comparing distances between hand keypoints identified in the two dimensional image and distances between corresponding hand keypoints in a two-dimensional projection of the three-dimensional parametric model. The loss/objective function further comprises a regularisation term penalising deviations from a predefined mean hand pose.

[0117] Based on the comparison, parameters of the three-dimensional hand model may be updated, for example using an optimisation procedure. Examples of such optimisation procedures include, Adam and/or other gradient descent methods.

[0118] At operation 7.4, said two-dimensional image and the corresponding parameters of the three-dimensional parametric hand model are stored in a computer memory. The two-dimensional image and the corresponding parameters of the three-dimensional parametric hand model may be stored as part of a training dataset, a validation dataset or a test dataset.

[0119] Operations 7.2 to 7.4 may be repeated for each of the extracted images (or a subset of the total extracted images). A subset of the extracted images satisfying some threshold condition may be used. For example, the threshold condition may be that a confidence score that the image contains an object of a given type (e.g. a hand) is above a threshold value (e.g. 90%).

[0120] The training set generated in this way may be used in any of the training methods described herein.

[0121] FIG. 8 shows a schematic example of a system/apparatus for performing any of the methods described herein. The system/apparatus shown is an example of a computing device. It will be appreciated by the skilled person that other types of computing devices/systems may alternatively be used to implement the methods described herein, such as a distributed computing system.

[0122] The apparatus (or system) 800 comprises one or more processors 802. The one or more processors control operation of other components of the system/apparatus 800. The one or more processors 802 may, for example, comprise a general purpose processor. The one or more processors 802 may be a single core device or a multiple core device. The one or more processors 802 may comprise a Central Processing Unit (CPU) or a Graphical Processing Unit (GPU). Alternatively, the one or more processors 802 may comprise specialised processing hardware, for instance a RISC processor or programmable hardware with embedded firmware. Multiple processors may be included.

[0123] The system/apparatus comprises a working or volatile memory 804. The one or more processors may access the volatile memory 804 in order to process data and

may control the storage of data in memory. The volatile memory 804 may comprise RAM of any type, for example Static RAM (SRAM), Dynamic RAM (DRAM), or it may comprise Flash memory, such as an SD-Card.

[0124] The system/apparatus comprises a non-volatile memory 806. The non-volatile memory 806 stores a set of operation instructions 808 for controlling the operation of the processors 802 in the form of computer readable instructions. The non-volatile memory 806 may be a memory of any kind such as a Read Only Memory (ROM), a Flash memory or a magnetic drive memory.

[0125] The one or more processors 802 are configured to execute operating instructions 808 to cause the system/apparatus to perform any of the methods described herein. The operating instructions 808 may comprise code (i.e. drivers) relating to the hardware components of the system/apparatus 800, as well as code relating to the basic operation of the system/apparatus 800. Generally speaking, the one or more processors 802 execute one or more instructions of the operating instructions 808, which are stored permanently or semi-permanently in the non-volatile memory 806, using the volatile memory 804 to temporarily store data generated during execution of said operating instructions 808.

[0126] Implementations of the methods described herein may be realised as in digital electronic circuitry, integrated circuitry, specially designed ASICs (application specific integrated circuits), computer hardware, firmware, software, and/or combinations thereof. These may include computer program products (such as software stored on e.g. magnetic discs, optical disks, memory, Programmable Logic Devices) comprising computer readable instructions that, when executed by a computer, such as that described in relation to FIG. 8, cause the computer to perform one or more of the methods described herein.

[0127] Any system feature as described herein may also be provided as a method feature, and vice versa. As used herein, means plus function features may be expressed alternatively in terms of their corresponding structure. In particular, method aspects may be applied to system aspects, and vice versa.

[0128] Furthermore, any, some and/or all features in one aspect can be applied to any, some and/or all features in any other aspect, in any appropriate combination. It should also be appreciated that particular combinations of the various features described and defined in any aspects of the invention can be implemented and/or supplied and/or used independently.

[0129] Although several embodiments have been shown and described, it would be appreciated by those skilled in the art that changes may be made in these embodiments without departing from the principles of this disclosure, the scope of which is defined in the claims.

1. A method, comprising:

receiving, as input to an embedding neural network that comprises one or more encoder parameters, a two-dimensional training image, wherein the two-dimensional training image comprises an object and is associated with a predetermined three-dimensional model that comprises vertices;

generating, using the embedding neural network, a first embedded representation of the two-dimensional training image;

- inputting the first embedded representation into a decoder model, the decoder model comprising a regression model that comprises one or more decoder parameters; generating, using the decoder model, a generated three-dimensional model of the object based on the first embedded representation, the generated three-dimensional model comprising vertices;
- comparing, using a loss function, the generated three-dimensional model of the object to the predetermined three-dimensional model associated with the two-dimensional training image; and
- updating the one or more encoder parameters and the one or more decoder parameters of the decoder model in dependence on the comparing.
2. The method of claim 1, further comprising:
- receiving, as input to the embedding neural network, a two-dimensional object image, wherein the two-dimensional object image comprises an image of the object; and
- generating, using the embedding neural network, a second embedded representation of a two-dimensional object image.
3. The method of claim 2, further comprising:
- inputting the second embedded representation of the two-dimensional object image into the decoder model that comprises the regression model; and
- generating, using the decoder model, a resulting three-dimensional model of the object from the second embedded representation, the resulting three-dimensional model comprising a plurality of nodes in a mesh; wherein the loss function comprises a vertex term and an edge term, the vertex term comprising first differences between vertices of the generated resulting three-dimensional model and vertices of the predetermined three-dimensional model, the edge term comprising second differences between edge lengths of the generated resulting three-dimensional model and edge lengths of the predetermined three-dimensional model.
4. The method of claim 3, wherein the vertices of the predetermined three-dimensional model of the object associated with the two-dimensional training image are derived by fitting a three-dimensional object model to the two-dimensional training image.
5. The method of claim 1, wherein the decoder model comprises a decoder neural network comprising one or more two-dimensional convolutional layers, and wherein the generated three-dimensional model corresponds to a hand, the generated three-dimensional model comprises three-dimensional coordinates of a plurality of nodes of a mesh.
6. The method of claim 1, wherein the decoder model comprises one or more upscaling layers.
7. The method of claim 1, wherein initial layers of the decoder model comprise a fully connected layer and a reshaping layer.
8. The method of claim 1, wherein the decoder model comprises a linear layer, and wherein the generated three-dimensional model corresponds to a hand, the generated three-dimensional model comprises three-dimensional coordinates of a plurality of nodes of a mesh.
9. The method of claim 1, further comprising:
- generating a training dataset for training the decoder model to generate a generated three-dimensional model of a deformable object from a two-dimensional object image, the generated three-dimensional model comprising generated parameters that comprise a plurality of nodes in a mesh; the generating the training dataset comprising:
- extracting a plurality of two-dimensional training images from one or more sections of video data, each two-dimensional training image of the plurality of two-dimensional training images comprising one or more deformable objects;
- for each two-dimensional training image in the plurality of two-dimensional training images:
- identifying, using a keypoint identification model, a plurality of object keypoints in the two-dimensional training image;
- determining parameters of a three-dimensional parametric model, the parameters corresponding to the deformable object in the two-dimensional training image, wherein determining the parameters comprises fitting the three-dimensional parametric model of the object to each two-dimensional training image using the identified plurality of object keypoints and corresponding object keypoints in the three-dimensional parametric model; and
- storing, in a computer memory, the two-dimensional training image and the corresponding parameters of the three-dimensional parametric model.
10. The method of claim 9, wherein the one or more sections of video data comprises one or more sections of video of use of sign language.
11. The method of claim 9, wherein fitting the three-dimensional parametric model of the object to each two-dimensional image comprises applying an iterative optimization procedure to an objective function,
- wherein the objective function comprises a reprojection term comparing locations of object keypoints identified in each two-dimensional training image with locations of corresponding object keypoints in a two-dimensional projection of the three-dimensional parametric model.
12. The method of claim 11, wherein the reprojection term comprises a mask that applies a weighting to the corresponding object keypoints in the two-dimensional projection.
13. The method of claim 12, wherein the objective function further comprises a length term comparing distances between object keypoints identified in each two-dimensional training image and distances between corresponding object keypoints in a two-dimensional projection of the three-dimensional parametric model.
14. The method of claim 13, wherein the objective function further comprises a regularization term penalizing deviations from a predefined mean object pose.
15. The method of claim 14, wherein the three-dimensional parametric model of the object is a mesh model parametrized by parameters comprising one or more shape parameters and/or one or more pose parameters.
16. The method of claim 15, wherein the parameters of the three-dimensional parametric model of the object further comprise one or more camera parameters.
17. The method of claim 16, wherein the corresponding object keypoints in the three-dimensional parametric model are identified from the mesh model using a sparse regression model.

18. A system comprising:
 at least one processor;
 a memory, the memory comprising computer readable instructions that, when executed by the at least one processor, cause the at least one processor to perform operations comprising:
 receiving, as input to an embedding neural network that comprises one or more encoder parameters, a two-dimensional training image, wherein the two-dimensional training image comprises an object and is associated with a predetermined three-dimensional model that comprises vertices;
 generating, using the embedding neural network, a first embedded representation of the two-dimensional training image;
 inputting the first embedded representation into a decoder model, the decoder model comprising a regression model that comprises one or more decoder parameters;
 generating, using the decoder model, a generated three-dimensional model of the object based on the first embedded representation, the generated three-dimensional model comprising vertices;
 comparing, using a loss function, the generated three-dimensional model of the object to the predetermined three-dimensional model associated with the two-dimensional training image; and
 updating the one or more encoder parameters and the one or more decoder parameters of the decoder model in dependence on the comparing.

19. A non-transitory computer readable storage medium comprising instructions that when executed configure hardware processing circuitry to perform operations comprising:

receiving, as input to an embedding neural network that comprises one or more encoder parameters, a two-dimensional training image, wherein the two-dimensional training image comprises an object and is associated with a predetermined three-dimensional model that comprises vertices;
 generating, using the embedding neural network, a first embedded representation of the two-dimensional training image;
 inputting the first embedded representation into a decoder model, the decoder model comprising a regression model that comprises one or more decoder parameters;
 generating, using the decoder model, a generated three-dimensional model of the object based on the first embedded representation, the generated three-dimensional model comprising vertices;
 comparing, using a loss function, the generated three-dimensional model of the object to the predetermined three-dimensional model associated with the two-dimensional training image; and
 updating the one or more encoder parameters and the one or more decoder parameters of the decoder model in dependence on the comparing.

20. The non-transitory computer readable storage medium of claim **19**, the operations further comprising:
 receiving, as input to the embedding neural network, a two-dimensional object image, wherein the two-dimensional object image comprises an image of the object; and
 generating, using the embedding neural network, a second embedded representation of a two-dimensional object image.

* * * * *